

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 06 -

MÉTRICAS DE AVALIAÇÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	5
SAIBA MAIS.....	9
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	26

EMENDAS

O QUE VEM POR AÍ?

Avaliar o desempenho de um modelo de Machine Learning pode parecer simples à primeira vista – muitas vezes recorre-se a uma única métrica como acurácia para dizer se um modelo “acertou muito”. No entanto, essa abordagem direta pode ser arriscada. Assim como lançar uma nova versão de software para todos os usuários de uma vez é um “salto no escuro”, confiar em apenas uma métrica de avaliação pode mascarar problemas graves. Por exemplo, imagine um classificador de spam que acerta 99% dos emails – parece ótimo, mas e se ele simplesmente marcar todos os emails como “não spam”? Teria alta acurácia (porque a maioria realmente não é spam), porém estaria deixando todo o spam passar despercebido! Esse tipo de situação ocorre quando usamos métricas inadequadas ou interpretamos de forma ingênua os resultados.

Felizmente, no campo de aprendizado de máquina, desenvolvemos um conjunto robusto de métricas de avaliação para diferentes situações, atuando como “canários” que nos alertam sobre falhas de um modelo antes que ele cause um estrago. Em vez de expormos todos os usuários a um modelo possivelmente ruim (analogia: deploy big bang), avaliamos o modelo em múltiplos critérios – por exemplo, olhando separadamente para precisão (proporção de predições positivas corretas) e revocação (proporção dos positivos reais que o modelo conseguiu capturar).

Assim, se o modelo de spam do exemplo estivesse ignorando muitos spams, a revocação baixa funcionaria como um alerta precoce, semelhante ao canário na mina sinalizando perigo. Podemos então iterar e melhorar o modelo antes de “promovê-lo” para uso geral. Em termos práticos, avaliar um modelo envolve escolher métricas adequadas ao problema e analisar os resultados de forma incremental e controlada – exatamente como uma implantação canário faz com novas versões de software.

Além disso, à medida que avançamos no estudo das métricas de avaliação, percebemos que cada uma delas ilumina um aspecto diferente do comportamento do modelo – e ignorar essas nuances pode levar a decisões equivocadas. Em disciplinas fundamentais de Machine Learning, entendemos que a escolha

adequada da métrica não é apenas um detalhe técnico, mas um passo estratégico: é ela que garante que avaliamos o modelo à luz do objetivo real do negócio, seja detectar fraudes raras, prever defeitos industriais ou classificar riscos médicos. Assim, aprender a selecionar e interpretar corretamente essas métricas é essencial para construir modelos confiáveis, robustos e realmente alinhados ao problema que se pretende resolver.

EMENDAS

HANDS ON

Vamos agora partir para a prática com um exemplo simples de cálculo de métricas de avaliação em Python. Suponha que desenvolvemos um modelo de classificação binária para detectar fraudes em transações. Temos um pequeno conjunto de resultados esperados (transações legítimas vs fraudadas), além das previsões feitas pelo nosso modelo, e queremos calcular as principais métricas: acurácia, precisão, revocação e F1-score. Depois, faremos o mesmo para um exemplo de regressão, calculando métricas como Erro Médio Absoluto (MAE), Erro Quadrático Médio (MSE) e sua raiz (RMSE), além do R^2 .

No código a seguir, definimos listas (ou tensores) de valores verdadeiros (y_{true}) e previsões do modelo (y_{pred}) para um problema binário. Em nosso exemplo, 1 indica “fraude” e 0 indica “não fraude”. Em seguida, calculamos a matriz de confusão – quantidade de Verdadeiros Positivos, Falsos Positivos, Verdadeiros Negativos e Falsos Negativos – e a partir dela derivamos as métricas:

```

1      # Dados de exemplo: 1 = fraude, 0 = nao fraude
2      y_true = [0, 0, 1, 0, 1, 1, 0, 0, 1, 0]
3      y_pred = [0, 0, 1, 0, 0, 1, 0, 1, 0, 0]
4
5      # Inicializar contadores da matriz de confusão
6      VP = FP = VN = FN = 0
7      for verdadeiro, previsto in zip(y_true, y_pred):
8          if previsto == 1 and verdadeiro == 1:
9              VP += 1 # Verdadeiro Positivo: previu 1,
era 1
10             elif previsto == 1 and verdadeiro == 0:
11                 FP += 1 # Falso Positivo: previu 1, era 0
12             elif previsto == 0 and verdadeiro == 0:
13                 VN += 1 # Verdadeiro Negativo: previu 0,
era 0
14             elif previsto == 0 and verdadeiro == 1:
15                 FN += 1 # Falso Negativo: previu 0, era 1
16
17         print(f"VP={VP}, FP={FP}, VN={VN}, FN={FN}\n")
18         # Saída esperada (por exemplo): VP=2, FP=1, VN=5,
FN=2

```

Código-fonte 1 - # Dados de exemplo: 1 = fraude, 0 = nao fraude (Python)

Fonte: Elaborado pelo autor (2026)

Rodando esse código, obtemos os valores da matriz de confusão. Com eles, podemos calcular as métricas de classificação básicas. A acurácia é a proporção de acertos do modelo (positivos e negativos): $\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$. A precisão (precision) é a fração de predições positivas que realmente eram positivas: $\text{Precision} = \frac{VP}{VP + FP}$. Já a revocação (recall, ou sensibilidade) é a fração de positivos reais que o modelo conseguiu identificar: $\text{Recall} = \frac{VP}{VP + FN}$. E o F1-score é a média harmônica entre precisão e revocação: $F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.

Vamos estender o código para calcular essas métricas usando os valores de VP, FP, VN, FN obtidos:

```

1      # (Continuando do código anterior: já temos VP, FP,
VN, FN definidos)
2      accuracy = (VP + VN) / (VP + VN + FP + FN)
3      precision = VP / (VP + FP) if (VP + FP) > 0 else 0.0
4      recall = VP / (VP + FN) if (VP + FN) > 0 else 0.0
5      f1 = 2 * (precision * recall) / (precision + recall)
if (precision + recall) > 0 else 0.0
6
7      print(f"Acurácia = {accuracy:.2f}\n")
8      print(f"Precisão = {precision:.2f}\n")
9      print(f"Revocação = {recall:.2f}\n")
10     print(f"F1-score = {f1:.2f}\n")

```

Código-fonte 2 - Estendendo o código para calcular essas métricas usando os valores de VP, FP, VN, FN obtidos (Python)

Fonte: Elaborado pelo autor (2026)

Nosso modelo de exemplo teria uma acurácia de, digamos, ~0.70 (70%). A precisão talvez seja algo como 0.67 (acertou 2 fraudes de 3 preditas) e a revocação 0.50 (identificou 2 de 4 fraudes existentes). Isso resultaria em um $F_1 \approx 0.57$. Esses números mostram que o modelo está longe do ideal – por exemplo, a revocação de apenas 50% indica que metade das fraudes passaram sem detecção (um alerta vermelho!). Já a precisão de 67% quer dizer que um terço das transações marcadas como fraude eram falsos alarmes.

Agora, para completar, vamos calcular métricas de erro para um problema de regressão. Suponha que temos um modelo que prevê o valor de um imóvel e queremos avaliar o quão próximas estão as previsões dos valores reais. Usaremos como métricas o MAE (erro médio absoluto), o MSE (erro quadrático médio) e o

RMSE (raiz quadrada do MSE), além do R^2 (coeficiente de determinação), que indica proporção da variância explicada (1.0 seria previsão perfeita):

```

1      import math
2
3      # Exemplo de valores reais (em milhares de dólares)
vs preditos pelo modelo
4      y_true_reg = [150, 200, 210, 180, 240] # preços
reais
5      y_pred_reg = [180, 195, 205, 170, 250] # previsões
do modelo
6
7      # Cálculo das métricas de regressão
8      n = len(y_true_reg)
9      # Erro Médio Absoluto (MAE)
10     mae = sum(abs(v - p) for v, p in zip(y_true_reg,
y_pred_reg)) / n
11     # Erro Quadrático Médio (MSE)
12     mse = sum((v - p)**2 for v, p in zip(y_true_reg,
y_pred_reg)) / n
13     # Raiz do Erro Quadrático Médio (RMSE)
14     rmse = math.sqrt(mse)
15     # Coeficiente de Determinação ( $R^2$ )
16     var_real = sum((v - sum(y_true_reg)/n)**2 for v in
y_true_reg)
17     r2 = 1 - (sum((v - p)**2 for v, p in
zip(y_true_reg, y_pred_reg)) / var_real)
18
19     print(f"MAE = {mae:.2f}\n")
20     print(f"MSE = {mse:.2f}\n")
21     print(f"RMSE = {rmse:.2f}\n")
22     print(f" $R^2$  = {r2:.3f}\n")

```

Código-fonte 3 - # Exemplo de valores reais (em milhares de dólares) vs preditos pelo modelo
(Python)

Fonte: Elaborado pelo autor (2026)

Suponhamos que o resultado foi: MAE = 16.0, MSE = 256.0, RMSE = 16.0 (nesse caso específico RMSE = MAE, mas em geral RMSE tende a penalizar mais os grandes erros) e $R^2 \approx 0,90$. Isso significa que, em média, o modelo erra o preço em 16 mil dólares e explica ~90% da variância dos preços reais. O R^2 próximo de 0.90 indica um bom desempenho geral para um primeiro modelo, mas lembrar que um R^2 alto não garante ausência de erros significativos – por isso olhamos também MAE/RMSE.

Por exemplo, um RMSE de 16 ainda pode ser significativo dependendo do contexto (16 mil dólares de erro pode ou não ser aceitável). Novamente, várias métricas nos dão um retrato mais completo do modelo: enquanto o R^2 foca na

variância explicada, o MAE nos dá uma noção clara em unidades do problema (dinheiro, no caso) do erro típico.

Com esse hands on, fica evidente como coleta e cálculo de múltiplas métricas permite julgarmos melhor um modelo. Assim como no deploy canário coletamos diferentes indicadores (erros, latência, uso de CPU) antes de decidir promover a versão nova, aqui coletamos diferentes métricas antes de dizer “este modelo está bom”.

No exemplo de classificação de fraudes, a acurácia sozinha poderia nos enganar (70% parece razoável), mas a análise conjunta com precisão/recall revelou falhas importantes. Já no exemplo de regressão, combinar RMSE e R^2 nos informa não apenas o erro médio, mas também o quão bem o modelo ajusta a variabilidade dos dados. Essa visão multifacetada é fundamental antes de “implantar” de fato um modelo em produção.

SAIBA MAIS

E a teoria? Até aqui vimos como fazer uma rede neural aprender, agora vamos entender por que e quando as redes neurais brilham, com um olhar mais teórico e rigoroso. Vamos dissecar formalmente o problema do aprendizado em múltiplas camadas e examinar as redes neurais por vários ângulos: limites de expressividade, algoritmos de treinamento, custos computacionais, formulação matemática da aprendizagem e boas práticas de desenvolvimento.

Formulação do problema

Treinar uma rede neural significa ajustar parâmetros (pesos e bias) para que uma função $f_{\mathbf{w}}(\mathbf{x})$ se aproxime do mapeamento desejado entre entradas e saídas. No caso do XOR, por exemplo, queremos $f_{\mathbf{w}}(0,1) \approx 1$ e outras combinações com suas respectivas saídas. Formalmente, temos um conjunto de pares $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ de treinamento e buscamos um conjunto de pesos $\mathbf{w}$ que minimize o erro total. Podemos pensar em um espaço de hipóteses onde cada configuração de pesos define uma função candidata; o treinamento é um processo de busca nesse espaço.

No perceptron de camada única, a função decidida é linear: $y = \theta(w_1x_1 + w_2x_2 + b)$, onde θ é uma função degrau (que retorna 1 se a soma ponderada excede um limiar, 0 caso contrário). Essa forma simples limita o que pode ser aprendido. Já um perceptron multicamada com funções de ativação não-lineares (como sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$) pode aproximar praticamente qualquer função contínua. Em termos matemáticos, redes neurais com pelo menos uma camada oculta e ativação não-linear são universais aproximadores – existe um teorema garantindo que, dado número suficiente de neurônios na camada oculta, a rede consegue aproximar qualquer função contínua com a precisão desejada. Porém, encontrar os pesos adequados (treinar a rede) é o grande desafio.

Linearmente separável vs não separável

Aqui reside um dos maiores marcos teóricos das redes neurais: um único neurônio (perceptron simples) consegue classificar apenas dados linearmente separáveis. Ou seja, ele traça um hiperplano que divide o espaço em duas regiões (saída 0 ou 1). Problemas como o XOR, mencionados no hands on, não são linearmente separáveis – não há uma linha única que separe perfeitamente as saídas 0 e 1 no plano. Isso foi destacado por Minsky e Papert em 1969, levando a um pessimismo temporário no campo (a chamada “primeira AI winter”).

A solução veio com a introdução das múltiplas camadas de neurônios e funções de ativação não-lineares nos nós ocultos. Com pelo menos uma camada oculta, as redes neurais conseguem combinar múltiplas fronteiras lineares para criar regiões de decisão não-lineares. No XOR, por exemplo, o MLP que usamos aprendeu essencialmente a separar as entradas em regiões via uma combinação de dois planos. Assim, superamos as limitações do perceptron simples. Em outras palavras: modelos lineares vs. modelos não-lineares – esta foi a virada que tornou as redes neurais tão poderosas. Sempre que uma única camada não dá conta (dados não separáveis linearmente), adicionamos camadas e neurônios até obter flexibilidade suficiente, pagando o preço de maior complexidade.

Algoritmos de treinamento e retropropagação do erro

Do ponto de vista algorítmico, treinar uma rede neural significa executar um ciclo iterativo de ajustes nos pesos para minimizar o erro geral. O algoritmo fundamental que viabilizou o treinamento de redes multicamadas é o da retropropagação do erro (backpropagation), popularizado nos anos 1980. Um ciclo básico de treinamento supervisionado pode ser visto como:

- **Inicialização:** definir os pesos $\mathbf{w}$ aleatoriamente (tipicamente valores pequenos) e escolher uma taxa de aprendizagem η (hiperparâmetro que controla o tamanho dos passos no ajuste).
- **Forward pass:** alimentamos a rede com um conjunto de entradas $\mathbf{x}$ e calculamos as ativações camada por camada até a saída $\hat{\mathbf{y}} = f_{\mathbf{w}}(\mathbf{x})$.

- **Cálculo do erro:** comparamos a saída $\hat{\mathbf{y}}$ com a saída esperada $\mathbf{y}$ usando uma função de custo (por exemplo, erro quadrático médio ou entropia cruzada). Obtemos um escalar $E(\mathbf{w})$ que indica o erro atual.
- **Retropropagação + ajuste:** computamos os gradientes $\frac{\partial E}{\partial \mathbf{w}}$, ou seja, como a variação de cada peso afeta o erro. Graças à regra da cadeia, o algoritmo backprop consegue calcular esses gradientes de forma eficiente, “propagando” o erro da última camada até as primeiras. Em seguida, ajustamos os pesos no sentido oposto ao gradiente: $\Delta w = -\eta \frac{\partial E}{\partial w}$. Em suma, diminuimos um pouco cada peso que contribuiu positivamente para o erro e aumentamos pesos que contribuíram negativamente, tentando reduzir E .
- **Repetição:** repetimos os passos de forward->erro->backprop múltiplas vezes (várias épocas), percorrendo muitos exemplos do dataset, até o erro total atingir um mínimo aceitável ou algum critério de parada (número de épocas, acurácia etc.).

Esse processo é denominado descida do gradiente (ou variações estocásticas/minibatch). Ele é determinístico no sentido do algoritmo (cada passo de ajuste é bem definido), porém, por envolver superfícies de erro complexas e inicialização aleatória, o resultado (pesos encontrados) pode não ser globalmente ótimo – às vezes a rede para em um mínimo local de erro. Técnicas como early stopping (parar o treinamento quando o erro de validação começa a piorar) agem como um “rollback” seguro: evitam overfitting interrompendo a aprendizagem no momento certo, similar à ideia de reverter uma mudança antes que cause danos. Assim como no Blue/Green tínhamos um rollback determinístico para voltar à versão estável, em redes neurais usamos validação e checkpoints de pesos para, se necessário, restaurar a melhor versão do modelo caso o treinamento comece a “desandar”.

Complexidade computacional e custo em redes neurais profundas

Redes neurais, por sua natureza, consomem considerável poder computacional durante o treinamento – especialmente as redes neurais profundas com muitas camadas. Em termos de uso de CPU/GPU e memória, o overhead pode crescer muito conforme aumentamos o número de neurônios e conexões. Por exemplo, uma rede simples com 100 neurônios em cada uma de 3 camadas densas pode ter milhares de pesos; já arquiteturas modernas (como redes com milhões de parâmetros) exigem hardware especializado.

Treinar um modelo de Deep Learning costuma escalar aproximadamente de forma linear com o volume de dados e com o número de parâmetros – ou seja, dobrar o dataset ou dobrar o tamanho da rede tende a dobrar o tempo de treinamento (complexidade $\mathcal{O}(N \cdot P)$, onde N é o número de exemplos e P , de parâmetros). No entanto, há retornos decrescentes: em clusters muito grandes, paralelizar o treinamento nem sempre traz aceleração perfeita, devido à comunicação entre nós. Ainda assim, comparado a abordagens que requerem intervenção manual, as redes neurais seguem viáveis em larga escala, suportadas por avanços em computação paralela.

Hoje, empresas utilizam clusters com GPUs ou TPUs para treinar modelos gigantes de visão ou linguagem em tempos aceitáveis. O custo financeiro, porém, é real: treinar um modelo de processamento de linguagem natural de última geração pode custar dezenas de milhares de dólares em infraestrutura de nuvem. Isso lembra o caso da estratégia Blue/Green em que manter dois ambientes duplicados é caro.

Similarmente, no mundo de IA, pesquisadores(as) e startups precisam equilibrar a ambição de arquiteturas profundas com custos de treino e tempo. Uma boa prática é dimensionar o modelo conforme a disponibilidade de dados e recursos computacionais – modelos muito grandes sem dados suficientes “superdimensionam” o problema, levando a desperdício e possivelmente overfitting. Por outro lado, modelos pequenos demais podem não capturar padrões (o chamado underfitting).

Uma curiosidade prática: assim como no Blue/Green às vezes se aquece o ambiente novo com tráfego sombra para evitar picos repentinos, em aprendizagem

profunda às vezes “aquecemos” a rede neural gradualmente – por exemplo, começando com uma taxa de aprendizagem alta e reduzindo aos poucos (estratégia de learning rate warm-up seguido de decaimento) para evitar mudanças bruscas que poderiam desestabilizar a convergência. Pequenos cuidados como esse garantem que “zero erro” não vire “explosão de gradiente” – ainda assim, bem melhor do que meses programando regras manualmente sem garantia de sucesso.

Modelagem matemática do aprendizado: função de custo e otimização

Podemos modelar o processo de aprendizado de uma rede neural de forma análoga ao que se faz para analisar risco em deploys Blue/Green. Em Blue/Green, definimos uma função de impacto $I(\Delta t)$ representando usuários afetados em função do tempo até detecção de problema e buscamos minimizar essa área. Em redes neurais, definimos uma função de perda $E(\mathbf{w})$ que quantifica o erro do modelo sobre os dados de treinamento. Por exemplo, para um conjunto de m exemplos com saídas y e previsões $\hat{y}$, poderíamos usar o erro quadrático médio:

$$E(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

O objetivo do treinamento é minimizar E tanto quanto possível, ajustando $\mathbf{w}$. A descida do gradiente mencionada é justamente uma técnica para encontrar um mínimo local dessa função de muitas variáveis. Matematicamente, idealmente gostaríamos do mínimo global, onde E é menor, indicando erro praticamente zero (modelo perfeito nos dados). Entretanto, devido à não-linearidade e alta dimensionalidade, $E(\mathbf{w})$ costuma ser uma superfície cheia de vales (mínimos locais) e montanhas (máximos). Surpreendentemente, na prática, algoritmos de otimização estocásticos conseguem achar soluções muito boas – talvez não o mínimo absoluto, mas suficientemente baixo para generalizar bem.

Outra peça importante da modelagem matemática é a probabilidade e a estatística: podemos interpretar a saída da rede (por exemplo, a probabilidade predita de uma classe) e o conjunto de dados sob uma ótica estatística. Redes neurais têm relação com modelos estatísticos clássicos – por exemplo, uma rede de uma camada com função sigmoide e loss de entropia cruzada é análoga a uma regressão logística. Já redes mais complexas correspondem a modelos não-lineares

difíceis de analisar teoricamente, mas ainda podemos usar conceitos como viés e variância para entender seu comportamento. A meta é minimizar não apenas o erro nos dados de treino, mas principalmente o erro em novos dados (generalização). Para isso, usam-se técnicas de regularização, que matematicamente adicionam termos à função de custo, penalizando pesos muito grandes, por exemplo. Isso equilibra a busca: minimizamos o erro mas também mantemos o modelo “simples” o suficiente, evitando sobreajuste.

Em resumo, o treinamento de redes neurais pode ser visto como um problema de otimização contínua de alta dimensão. Encontrar soluções exige tanto heurísticas algorítmicas (como backprop + SGD) quanto truques matemáticos (como funções de custo adequadas, normalização dos dados, inicialização inteligente dos pesos etc.). Felizmente, décadas de pesquisas forneceram um conjunto robusto de ferramentas para tratar esses aspectos, tornando o treinamento de redes algo sistemático – assim como o Blue/Green, que inicialmente parecia arriscado, acabou ganhando procedimentos bem definidos para garantir sucesso.

Arquiteturas e versões de redes neurais

No contexto de redes neurais, assim como no de software, existem versões e topologias diversas de modelos e grandes mudanças arquiteturais requerem cautela adicional. Enquanto no Blue/Green falamos de versões “v1.x” vs “v2.0” de um sistema, nas redes neurais podemos ter a “versão 1” de um modelo (digamos, uma rede rasa) e a “versão 2” (uma rede profunda aprimorada). Atualizar um modelo para uma arquitetura nova envolve considerações similares a um deploy: precisamos garantir que a nova versão compatibiliza com dados e requisitos. Muitas empresas optam, por exemplo, por lançar gradualmente um modelo de IA novo em produção – primeiro para uma pequena porcentagem de usuários (um canary release de modelo) e, se tudo correr bem, expandem para 100%. Trata-se basicamente de aplicar Blue/Green e feature flags no mundo de modelos.

Falando em arquiteturas, as redes neurais se subdividem em muitos tipos: perceptrons de camada única, redes multicamadas (MLPs), redes convolucionais (CNNs), redes recorrentes (RNNs), redes transformer, entre outras. Cada arquitetura traz uma proposta diferente (por exemplo, CNNs introduzem

camadas de filtros para visão, RNNs possuem conexões de feedback para sequências, Transformers usam mecanismos de atenção etc.). Na prática, a evolução de versões de modelos muitas vezes significa migrar para uma arquitetura mais avançada. Foi o caso clássico da visão computacional: modelos “v1” pré-2012 usavam muito SVMs com features feitas à mão; então veio a “v2” com redes neurais profundas (AlexNet e sucessores) que revolucionaram o campo com enormes ganhos de acurácia.

Essa transição não foi trivial – exigiu coletar muito mais dados, usar GPUs e adotar novas técnicas. Equipes tiveram que garantir que a nova geração de modelos era compatível com os sistemas existentes (formatos de input/output, tempos de resposta etc.), fazendo períodos de testes paralelos. Em essência, mantiveram o modelo antigo em produção enquanto validavam o novo em sombra, semelhante ao Blue/Green, até que a confiança fosse suficiente para o cutover. Hoje, convive-se com múltiplos modelos: algumas aplicações mantêm uma versão simplificada operando em dispositivos móveis enquanto outra, mais pesada, roda na nuvem para maior precisão – uma espécie de Rainbow Deployment de modelos, cobrindo diferentes trade-offs de performance.

Assim como versionamento semântico impõe disciplina para compatibilidade, no aprendizado de máquina empregamos disciplina ao mudar a arquitetura: nunca remover funcionalidades aprendidas que eram essenciais sem ter alternativa (análoga à compatibilidade de API). Por exemplo, se uma versão nova de um modelo de recomendação “esquece” de tratar um tipo de item que a versão anterior recomendava bem, usuários notarão regressão – equivalente a um deploy Green com quebra de contrato. Por isso, equipes de ML fazem extensivos testes A/B, monitoram métricas de negócio e muitas vezes mantêm em produção ambos os modelos (antigo e novo) atendendo partes do tráfego, só desativando o antigo após plena confiança (novamente, igual ao Blue/Green gradual).

Em termos de topologia de modelos, podemos desenhar redes neurais de várias formas: desde estruturas bem divididas em duas partes (por exemplo, um encoder e um decoder – análogo a ativo/passivo cooperando) até ensembles de vários modelos diferentes votando em conjunto (vários ativos simultâneos, como um Rainbow de algoritmos). Cada topologia tem suas vantagens: ensembles tendem a aumentar acurácia e robustez (cada modelo cobre fraquezas de outro), mas

custam mais para servir. Embora o estado da arte em diversas áreas hoje seja dominado por mega redes neurais (um único modelo enorme dominando a tarefa), há tendências de ter não apenas um modelo mas vários modelos especializados atuando em conjunto – por exemplo, em um assistente virtual, um modelo de ASR (voz->texto), outro de NLP para intenção, outro de síntese de voz. Isso mostra que, mesmo em IA, às vezes é preferível orquestrar múltiplos componentes (cada um análogo a um microserviço “verde/azul”) do que jogar tudo em um monolito gigante. A chave é garantir que esses componentes comuniquem-se bem e permaneçam compatíveis conforme evoluem.

Controle de experimentos e reproducibilidade de modelos

Por fim, vale conectar o desenvolvimento de redes neurais à governança e boas práticas de engenharia, assim como fizemos com Blue/Green em pipelines GitOps. Em equipes grandes de ciência de dados e engenharia de ML, nem todos podem ou devem treinar e implantar modelos criticamente importantes sem supervisão – costuma-se definir via permissões (similar ao RBAC, controle de acesso baseado em papéis) quem pode aprovar o deploy de um novo modelo em produção.

Ferramentas de MLOps – o “DevOps da aprendizagem de máquina” – como MLflow, Kubeflow ou TensorFlow Extended (TFX) permitem gerenciar experimentos, versionar modelos e automatizar a implantação de forma controlada. Por exemplo, mantém-se em um repositório não só o código, mas também a definição da arquitetura e até o snapshot dos pesos treinados (um arquivo de modelo). Quando uma nova versão do modelo é treinada e validada, pode-se acionar um pipeline CI/CD que registra esse modelo em uma base de modelos e, após aprovação, atualiza o serviço de inferência em produção para usar a nova versão – conceitualmente idêntico a promover o ambiente Green a ativo.

Uma vantagem clara de se ter esse processo automatizado e versionado é a auditabilidade – cada modelo em produção está associado a um experimento reprodutível (com número de versão, hiperparâmetros, dataset utilizado etc.), facilitando compliance e post-mortems em caso de problemas. Além disso, essas ferramentas enfatizam configurações idempotentes: por exemplo, se rodarmos

novamente o pipeline de treinamento com a mesma semente aleatória e dados, devemos obter o mesmo modelo (ou muito semelhante). Para isso, fixamos random seeds, documentamos bibliotecas e versões e até armazenamos o conjunto de dados de treino usado. Provar formalmente a idempotência em aprendizado de máquina é complicado (devido à aleatoriedade inerente e não-convexidade do problema), mas na prática conseguimos um alto grau de reprodutibilidade determinística se seguirmos as boas práticas.

A filosofia de MLOps combina bem com as lições do Blue/Green: define-se claramente “ambientes” ou stages para os modelos (ex.: staging vs. produção) e promove-se um modelo de um stage a outro apenas via pipeline controlado e com validação. Muitas empresas implementam avaliações manuais no fluxo de aprovação de modelos: o pipeline treina o modelo “verde” e o deixa pronto em staging, mas só atualiza o serviço de produção quando um especialista revisa e dá o OK.

Essa camada extra de verificação humana é importante em sistemas críticos – afinal, um modelo de IA mal treinado em produção pode ser tão desastroso quanto um deploy bugado. Em suma, o campo de redes neurais aprendeu a importância de governança e reprodutibilidade: tão crucial quanto inventar arquiteturas brilhantes é conseguir replicar o experimento, comparar resultados e reverter a tempo caso necessário. Com essas ferramentas e práticas, a implantação de modelos de IA torna-se tão confiável quanto os deploys de aplicações tradicionais – integrando-se de forma natural ao ciclo de desenvolvimento de software.

MERCADO, CASES E TENDÊNCIAS

A escolha adequada de métricas e a interpretação correta dos resultados são cruciais na indústria – tão cruciais quanto estratégias de implantação segura. Vamos ver alguns casos reais (e um hipotético) que ilustram lições sobre métricas de avaliação e discutir para onde caminha essa área.

		Classe positiva	Classe negativa
Classe prevista	Classe positiva	VP	FP
	Classe negativa	FN	VN

Confusão de confusão

Figura 1 – Matriz de confusão ilustrativa de um modelo de classificação binária, mostrando contagens de VP, FP, VN, FN
Fonte: Google Imagens (2026)

No quadro da Figura 1, visualizamos a matriz de confusão de um modelo e algumas métricas derivadas. Esse tipo de representação é amplamente utilizado em empresas para monitorar o desempenho de modelos em produção. Por exemplo, a equipe de um sistema de detecção de fraudes pode acompanhar diariamente a matriz de confusão das transações: se notarem um aumento repentino de falsos negativos (fraudes não detectadas), isso acende um alerta para revisar o modelo ou os dados – tal como um dashboard de deploy canário acenderia se a taxa de erro subisse.

Muitas organizações estabelecem limiares de alerta nas métricas: “Precisão caiu abaixo de X%” ou “Revocação diária abaixo de Y%” disparam investigação imediata, similar ao monitoramento de SLOs de serviço. Esse acompanhamento proativo de métricas garante que um modelo degradado não passe despercebido, do

mesmo modo que um canário em produção detecta regressão de performance antes que cause um grande impacto.

Um caso clássico envolvendo métricas ocorreu no Netflix Prize, uma competição de 2006-2009 em que a Netflix ofereceu 1 milhão de dólares para quem melhorasse em 10% a RMSE das previsões de ratings de filmes em relação ao seu algoritmo existente. A métrica escolhida – RMSE – guiou milhares de equipes pelo mundo. Em 2009, o time vencedor conseguiu ~10.05% de melhoria no RMSE com um algoritmo extremamente complexo (ensemble de dezenas de modelos). Curiosidade: a Netflix, porém, nunca implantou esse modelo vencedor na íntegra. Por quê? Embora o RMSE tenha de fato melhorado, as diferenças em métricas de negócio (como engajamento de usuários) não foram significativas e o ganho marginal não justificou a enorme complexidade adicionada.

Esse caso ensinou ao mercado que melhorar uma métrica de avaliação não garante sucesso do produto – é preciso alinhar as métricas técnicas com métricas de negócio. A Netflix passou a se preocupar mais com métricas como acurácia de recomendações percebida pelos usuários e não apenas um erro médio global. Ainda assim, o Netflix Prize revolucionou a área porque estabeleceu um padrão quantificável (RMSE) que estimulou inovação. Muitas empresas seguiram com competições similares no Kaggle, sempre definindo de antemão a métrica que os competidores devem otimizar. A lição: escolher bem essa métrica é vital, pois “você recebe aquilo que mede” – se medir RMSE, os times vão otimizar RMSE, mas talvez à custa de outras qualidades não medidas.

Falando em Kaggle, a plataforma de competição de Data Science popularizou diversas métricas e conscientizou a comunidade sobre sutilezas. Por exemplo, competições de classificação por vezes utilizam AUC-ROC como métrica alvo. Houve casos em que líderes na métrica AUC tinham piores métricas de precisão/recall em regimes de interesse prático. Isso porque otimizar AUC pode levar um modelo a ser muito bom em ordenar exemplos (separar positivos de negativos globalmente), mas não necessariamente bom num ponto específico da curva PR que seja relevante. Em problemas de busca e ranking, métricas como NDCG (Normalized Discounted Cumulative Gain) ou MAP (Mean Average Precision) são empregadas, pois importam mais a ordenação dos top resultados do que uma taxa bruta de acerto. Empresas aprendem com isso a definir métricas customizadas

para seus casos: por exemplo, um e-commerce avalia recomendações pelo CTR ponderado (click-through rate dando mais peso a clicks em produtos de maior interesse) em vez de acurácia pura de clique.

Vejamos um cenário fictício que espelha erros reais comuns: uma startup de segurança desenvolve um detector de intrusão em redes. Na pressa de mostrar eficácia, divulga que seu modelo tem 99.9% de acurácia. Impressionante? Descobre-se depois que menos de 0.1% dos eventos de rede eram ataques, e o modelo praticamente ignorou todos os ataques (classificando tudo como tráfego normal) – ou seja, fracasso total, mas mascarado por um valor inflado de acurácia. Clientes ficam insatisfeitos e a reputação da startup é abalada. Esse exemplo hipotético reflete situações verídicas onde profissionais inexperientes escolhem a métrica errada. Hoje, é consenso na indústria que, para esses casos de detecção rara, métricas como revocação no positivo (também chamada sensibilidade) e a taxa de falso alarme (1 - precisão, ou especificidade do negativo) devem ser o foco. Muitos contratos de fornecimento de sistemas de detecção incluem no SLA algo como: “O modelo deve manter sensibilidade ≥ 0.95 e especificidade ≥ 0.99 ” – garantindo poucos falsos negativos e falsos positivos. Note a pluralidade: raramente uma única métrica basta como requisito.

Grandes empresas tech mantêm catálogos de métricas e guias de melhores práticas. O Google, por exemplo, enfatiza avaliar classificadores de busca tanto em métricas offline (precisão de resultados) quanto em métricas online (satisfação do usuário medida via intervenções A/B). O Facebook relatou que para sistemas de detecção de conteúdo abusivo, treinava modelos otimizando AUC-ROC, mas definia thresholds operacionais selecionando pontos da curva PR que correspondiam a níveis aceitáveis de precisão. Ou seja, usava AUC para escolher modelos candidatos e precision/recall para calibrar a decisão final.

Outra tendência clara é a incorporação de MLOps (Machine Learning Ops) no pipeline de métricas. Assim como o DevOps trouxe automação para deploys com canário, o MLOps traz automação para avaliação contínua de modelos. Empresas como Uber e Airbnb implementaram sistemas de monitoramento que calculam métricas em tempo real no modelo em produção e as comparam com referências históricas. Se um desvio significativo ocorre – por exemplo, a acurácia medida nos últimos 1000 casos caiu 5 pontos percentuais em relação à semana anterior – o

sistema pode automaticamente disparar um processo de retraining ou acionar engenheiros. Esse é o análogo, para modelos, do rollback automatizado em canário quando a métrica cai. Assim, a fronteira entre avaliação e produção está se esvaindo: modelos aprendem continuamente, e suas métricas viram parte do observability do sistema.

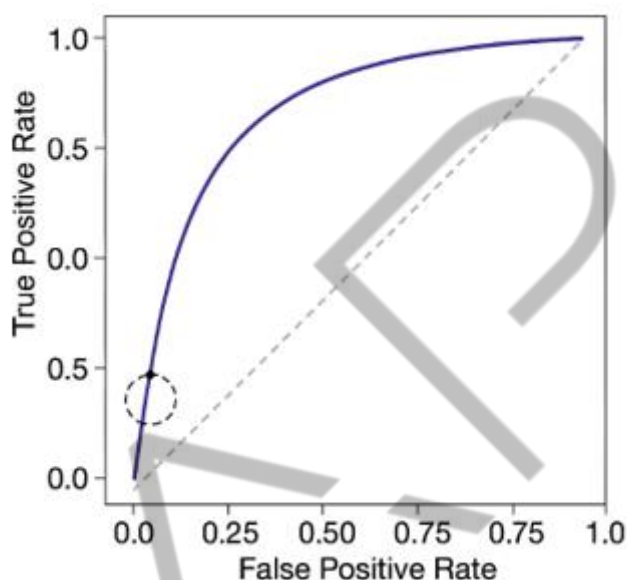


Figura 1 – Exemplo de curva ROC (a linha vermelha representa o desempenho de um modelo varrendo diferentes limiares; o ponto tracejado indica um possível limiar operativo)
Fonte: Google Imagens (2026)

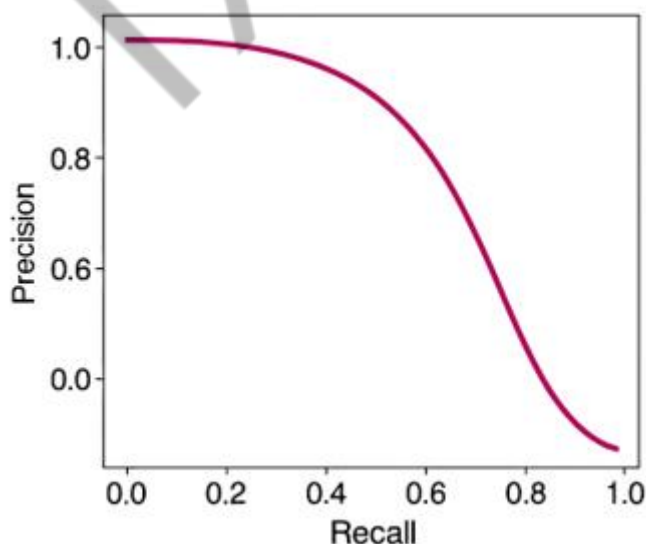


Figura 2 – Exemplo de curva Precisão vs. Revocação para o mesmo modelo da figura 1
Fonte: Google Imagens (2026)

Os gráficos 1 e 2 ilustram a diferença de perspectiva entre ROC e PR. No Gráfico 1, nosso modelo (linha roxa) apresenta um excelente desempenho geral – a área sob a curva ROC é alta, e no ponto operativo sugerido (círculo pontilhado) temos cerca de 90% de verdadeiros positivos para apenas 5% de falsos positivos. Já no Gráfico 2, vemos que ao alcançar revocação próxima de 90%, a precisão despenca para perto de 40%. Isso indica que, embora o modelo consiga pegar a maioria dos positivos, ele o faz ao custo de muitos alarmes falsos. Em um cenário real de fraude, esse ponto poderia significar sobrecarga na equipe de analistas verificando muitos falsos alertas. Por outro lado, se operarmos o modelo num limiar mais rigoroso para elevar a precisão (digamos, ponto em que precisão ~ 0.80), o gráfico PR mostra revocação caindo para algo em torno de 60%. Esse empasse – aumentar precisão diminui revocação e vice-versa – é fundamental no uso prático de classificadores. Empresas de fintech, por exemplo, muitas vezes preferem um equilíbrio mais voltado à revocação máxima (não perder fraudes) tolerando mais falsos alertas, enquanto em recomendação de produtos pode-se preferir alta precisão (mostrar só resultados relevantes) desistindo de cobrir todos os itens. Assim, ambos os gráficos são usados em conjunto para escolher o melhor compromisso de acordo com os objetivos de produto, algo que uma métrica única não evidência por completo.

Futuro das métricas – MLOps autônomo e AutoML

Olhando adiante, vemos as práticas de avaliação de modelos se entrelaçando cada vez mais com fluxo de desenvolvimento e operação automatizada. No paradigma de AutoML, por exemplo, a seleção de modelos e de hiperparâmetros é guiada por métricas: o sistema treina múltiplos modelos e escolhe o campeão segundo a métrica definida (ex: maior AUC, menor MSE). Tendências recentes levam isso além – envolvendo também múltiplas métricas. Já existem abordagens multiobjetivo em que se otimiza uma combinação, por exemplo, maximizar AUC-ROC e minimizar o tempo de inferência. Algoritmos de bandits e aprendizado por reforço vêm sendo aplicados para ajustar dinamicamente alguns parâmetros de modelo após implantação, visando manter as métricas dentro de faixas desejadas. Por exemplo, um classificador em produção pode lentamente ajustar seu limiar de decisão usando um bandit bayesiano que experimenta leves aumentos ou reduções

e observa impacto em precisão/revocação, convergindo para um ponto ótimo de F1 ao longo do tempo – de forma autônoma.

Outra forte tendência é incorporar metamétricas de confiança na avaliação. Não basta reportar “Modelo teve 95% de acurácia”, mas também a incerteza dessa acurácia e em quais regiões de dados ele performou pior. Isso envolve técnicas de análise de erro mais sofisticadas e validação cruzada. Em lugar de um número estático, equipes estão passando a fornecer dashboards ricos: distribuição das probabilidades preditas do modelo (para ver se está calibrado), curva de ganho cumulativo (para ver o retorno se agirmos sobre top X predições), etc.

No contexto de Responsabilidade e IA, embora fugir um pouco do escopo estritamente “intro/intermediário”, vale mencionar que a definição de sucesso de um modelo começa a incluir métricas de justiça e explicabilidade. Por exemplo, grandes empresas ao lançar um modelo de crédito não olham apenas AUC e KS (Kolmogorov-Smirnov), mas também métricas de viés entre grupos (como diferença de revocação para diferentes etnias). Ainda que métricas de fairness sejam um tópico avançado, a tendência é clara: o “sucesso” de um modelo será medido por um painel multifacetado de métricas, e não por um único número mágico.

Por fim, a integração entre monitoramento de métricas e pipeline de entrega está se fechando em loop. Ferramentas de CI/CD específicas para ML (como AWS SageMaker Model Monitor, Azure ML Monitoring etc.) permitem disparar automaticamente um novo ciclo de treinamento ou ajuste de um modelo se as métricas em produção degradarem além de certo limite – similar à ideia de rollback automático. Isso significa que, no futuro próximo, poderemos ter um sistema de deploy de modelos que atua como um canário contínuo: ele libera o modelo, observa as métricas (tanto offline nos testes quanto online em uso real), e decide de forma autônoma continuar ou retroceder, ou até treinar um modelo alternativo, sem intervenção humana imediata. Em resumo, as métricas de avaliação, outrora vistas apenas como “notas de um modelo”, estão se tornando atores centrais no ciclo de vida de sistemas de IA – guiando decisões automáticas, alinhando modelos a objetivos de negócio e garantindo (assim esperamos) que as IAs sejam implantadas com a mesma prudência com que fazemos deploy de código em produção.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os Fundamentos de Métricas de Avaliação de modelos de Machine Learning, entendendo por que escolher a métrica correta é tão importante quanto treinar um bom modelo. Logo no início, discutimos os perigos de olhar apenas a acurácia – assim como um deploy “big bang” pode mascarar problemas, uma única métrica pode esconder falhas graves do modelo. Vimos que métricas servem como “canários” no desenvolvimento de modelos, alertando sobre desempenho insatisfatório antes que ele cause danos (por exemplo, detectar um classificador de fraude que pega poucas fraudes apesar de alta acurácia). No Hands On, calculamos na prática diversas métricas de classificação (precisão, revocação, F1) e de regressão (MAE, MSE, RMSE, R^2) para exemplos fictícios, mostrando como cada uma revela um aspecto do erro do modelo – e aprendemos a implementá-las manualmente em Python/PyTorch, reforçando seus significados.

Em seguida, no Saiba Mais, formalizamos esses conceitos: definimos matematicamente as métricas básicas e discutimos casos de uso apropriados (por exemplo, porque usar acurácia balanceada em conjuntos desbalanceados). Introduzimos a noção de testes estatísticos para comparação de modelos – garantindo que melhorias de 1-2% sejam reais e não ruído – e de intervalos de confiança para entender a robustez das métricas medidas. Aprofundamos nas curvas ROC e Precisão-Revocação, entendendo como elas oferecem uma visão de todo o espectro de performance do classificador conforme variamos limiares, e como escolher um ponto ótimo de operação de acordo com as necessidades do negócio. Também abordamos métricas de regressão e destacamos a importância de avaliar não só a qualidade do ajuste (R^2) mas também o erro em unidades do problema (MAE/RMSE). Ao longo da teoria, traçamos paralelos com implantação de software: monitorar métricas de modelo ao longo do tempo é análogo a monitorar métricas de sistema em um canário, e definir critérios de sucesso para promover um modelo é similar a gates em deploy contínuo.

Na parte de Mercado, Cases e Tendências, analisamos exemplos concretos: relembramos o Netflix Prize e como otimizar apenas RMSE nem sempre se traduz em valor de produto, enfatizando o alinhamento de métricas técnicas com métricas de negócio. Vimos (hipoteticamente) que vangloriar-se de acurácia sem contexto

pode ser desastroso – um modelo pode parecer bom e na prática falhar completamente por causa de dados desbalanceados. Discutimos que empresas líderes combinam várias métricas e até criam métricas customizadas para refletir melhor seus objetivos (como CTR ponderado, NDCG em rankings, etc.). Apresentamos gráficos comparativos de ROC vs PR que ilustram a importância de entender trade-offs: não existe modelo perfeito, e sim pontos de operação preferíveis dependendo se queremos evitar mais falsos positivos ou falsos negativos. Por fim, debatemos tendências como o uso de MLOps para monitoramento contínuo de métricas (e gatilhos automáticos de retreinamento/rollback), a ascensão de AutoML e algoritmos que otimizam modelos com mínima intervenção humana (mas que exigem definição cuidadosa da métrica objetivo), e a perspectiva de métricas mais amplas incorporando aspectos de equidade e explicabilidade, indicando que o conjunto de “níveis de qualidade” de um modelo está crescendo.

Em resumo, você aprendeu que as métricas de avaliação são a bússola que guia todo o ciclo de vida de um modelo de Machine Learning. Saber escolher e interpretar essas métricas faz a diferença entre implantar um modelo confiável que trará ganhos e um modelo que pode falhar silenciosamente. Com esse conhecimento, você está apto a identificar quais métricas importam para cada tipo de problema, calcular e analisar resultados de forma crítica e adotar as melhores práticas do mercado na avaliação e monitoramento de modelos de IA.

REFERÊNCIAS

BISHOP, C. M. **Pattern Recognition and Machine Learning**. 2006. Disponível em: Springer – PRML. Acesso em: 18 jan. 2026.

BROWNLEE, J. **Classification Accuracy is Not Enough: More Performance Measures You Can Use**. 2019. Disponível em: <https://machinelearningmastery.com/classification-accuracy-is-not-enough-more-performance-measures-you-can-use/>. Acesso em: 18 jan. 2026.

CHAI, T.; DRAXLER, R. Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE. **Geoscientific Model Development**, v. 7, n. 3, p. 1247–1250, 2014. Disponível em: <https://gmd.copernicus.org/articles/7/1247/2014/gmd-7-1247-2014.html>. Acesso em: 18 jan. 2026.

CZAKON, J. **F1 Score vs ROC AUC vs Accuracy vs PR AUC: Which Evaluation Metric Should You Choose?** 2025. Disponível em: <https://neptune.ai/blog/f1-score-accuracy-roc-auc-pr-auc>. Acesso em: 18 jan. 2026.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**. 2. ed. Sebastopol, CA: O'Reilly Media, 2019.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge, MA: MIT Press, 2016.

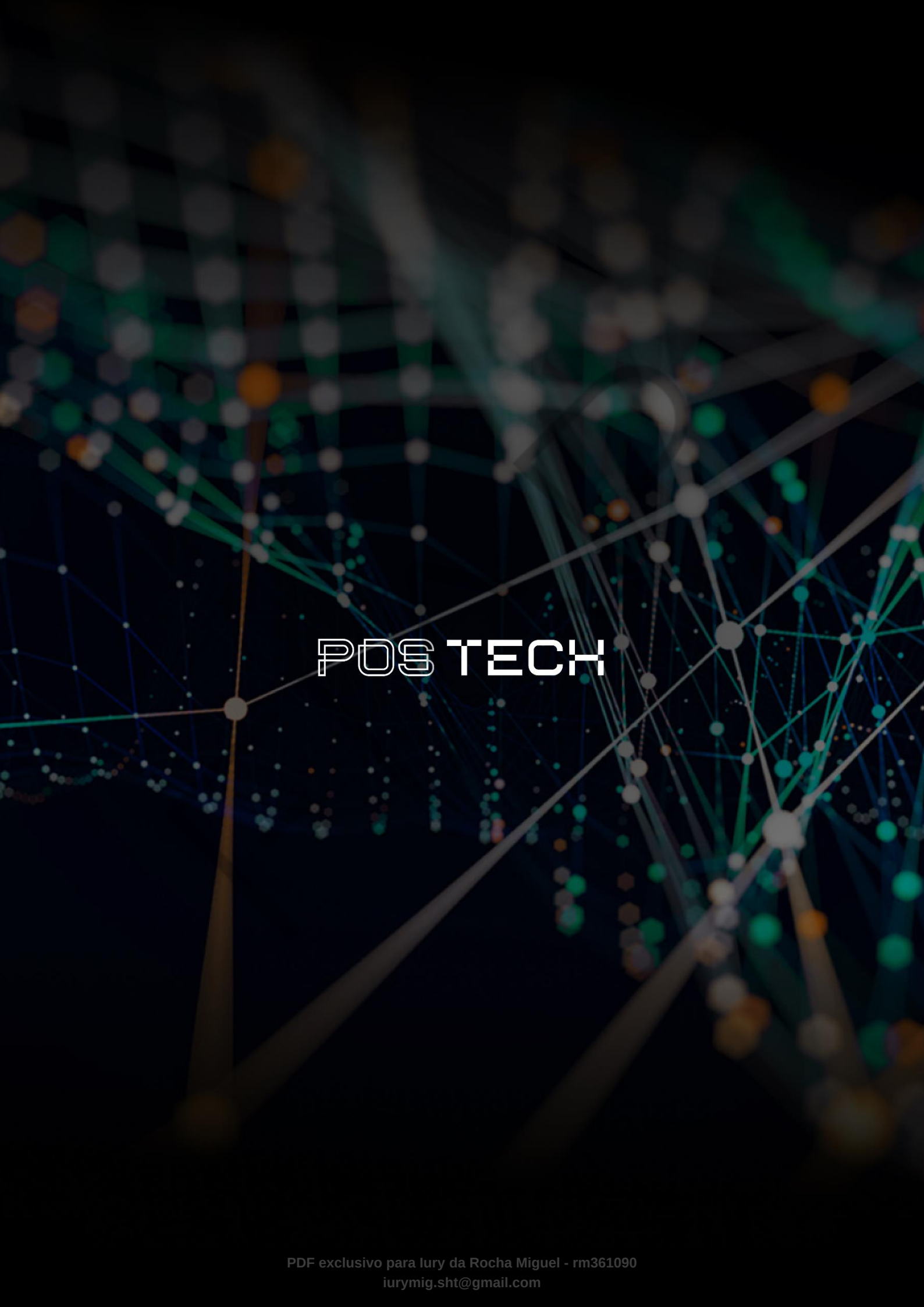
POWERS, D. M. Evaluation: From Precision, Recall to ROC, Informedness, Markedness & Correlation. **Journal of Machine Learning Technologies**, v. 2, n. 1, p. 37–63, 2011. Disponível em: <https://arxiv.org/abs/2010.16061>. Acesso em: 18 jan. 2026.

SAITO, T.; REHMSMEIER, M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. **PLoS ONE**, v. 10, n. 3, e0118432, 2015. Disponível em: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0118432>. Acesso em: 18 jan. 2026.

PALAVRAS-CHAVE

Acurácia. Precisão. RMSE.

EMSE



POSTECH